Session 6
AADK Task 6

Assessment Report on
Jetpack Compose Button Interaction & State Management Report

Name : Pawankumar

Course : Android App Development with Gen AI

Jetpack Compose Button Interaction & State Management Report

Table of Contents

1	Abstract
2	Introduction
3	System Architecture Diagram
4	Task 1 – Button Interaction Fundamentals & Analysis
5	Task 2 – Designing Interactive Screen with Multiple Buttons & States
6	Task 3 – Usability Review & Accessibility for Interactive Buttons
7	Task 4 – Interactive Feature Plan & Implementation Report
8	Detailed Code Explanation
9	Conclusion
10	References & Bibliography

Abstract

This report explains interactive user interface development using Jetpack Compose. It focuses on button interactions, state management, usability, accessibility, and feature planning. The document is organized into multiple tasks which demonstrate how user actions trigger program logic and update UI dynamically through recomposition.

Introduction

Interactive user interfaces are essential in modern mobile applications. Buttons allow users to trigger operations such as navigation, state changes, and data updates. Jetpack Compose simplifies UI development by using a declarative approach where UI automatically updates when

System Architecture Diagram

The architecture demonstrates the flow of interaction from user input to UI update. When a user taps a button, the click event triggers a function which updates a state variable. Jetpack Compose observes this state change and recomposes the affected UI components.

Task 1
Button Interaction Fundamentals & Analysis

Task 1 focuses on understanding how button interaction works in Jetpack Compose. When a user taps a button, the onClick event handler executes logic that modifies application state.

When a user presses a button, the system executes a function defined in the **onClick handler**. This function modifies the application's state. Once the state changes, Compose automatically recomposes the affected UI components and updates what the user sees on the screen.
This process follows a simple interaction cycle:
User Action → Event Handler → State Change → UI Recomposition
Understanding this cycle is essential because most mobile applications rely heavily on interactive UI components such as buttons, toggles, switches, and sliders.

Example Compose Code

```
var count by remember { mutableStateOf(0) }
Button(onClick = { count++ }) {
   Text("Increment")
}

Text("Count: $count")
```

Explanation of Code

1. remember { mutableStateOf(0) }

This line creates a **state variable** called count.

- mutableStateOf() creates observable state.
- remember ensures the value survives recomposition.

2. Button Composable

Button(onClick = { count++ })
This defines a button component. When the button is clicked, the function inside onClick runs.

3. State Update

count++
This increases the value of count.

4. UI Recomposition

Text("Count: $count")
Because count is a state variable, Compose detects the change and **recomposes the UI**, updating the text automatically.

Reflection

Designing interactive UI components can present several challenges:

1. **Managing State Properly**
    - If state is not handled correctly, the UI may show incorrect values.
2. **Handling Frequent User Actions**
    - Rapid button clicks may cause unexpected behavior.
3. **Maintaining Clean UI Logic**
    - Complex UI logic can make code difficult to maintain.

Developers solve these challenges by using **proper state management techniques and modular UI design**.

Task 2
Designing Interactive Screen with Multiple Buttons & States

Button	Action	State Change	UI Result
Add	Increase value	count++	Number increases
Remove	Decrease value	count--	Number decreases
Reset	Reset state	count=0	Value resets

In multi-button interfaces each button modifies application state differently. Proper state modeling ensures the UI remains consistent and predictable.

Real-world applications often include multiple actions on the same screen. For example:

- Add item
- Remove item
- Reset values
- Submit data

Each button performs a different operation, and the UI must update correctly based on the action performed.

UI Layout Design

The UI contains:

- A **title text**
- A **display text showing the counter value**
- **Three buttons (Add, Remove, Reset)**

Example layout structure:

```
Column
├── Text("Counter App")
├── Text("Count: $count")
├── Button(Add)
├── Button(Remove)
└── Button(Reset)
```

Importance of State Modeling

When multiple buttons modify state variables, developers must clearly define **state transitions**.

Example state flow:
Count = 0
Press Add → Count = 1
Press Add → Count = 2
Press Remove → Count = 1
Press Reset → Count = 0
Proper state modeling ensures:

- UI consistency
- Predictable behavior
- Easier debugging

Reflection

Without proper state management, several issues may occur:

- Incorrect values displayed
- Buttons affecting the wrong state
- UI becoming inconsistent

Testing on **different screen sizes and device orientations** helps ensure the UI behaves correctly.

Task 3
Usability Review & Accessibility for Interactive Buttons

Description

Task 3 focuses on evaluating the usability and accessibility of interactive buttons. A well-designed user interface must not only function correctly but also be easy to use for all users, including those with disabilities.

Accessibility ensures that applications can be used by people with:

- **Visual impairments**
- **Motor disabilities**
- **Cognitive challenges**

Accessibility ensures all users including visually impaired individuals can interact with the application. Compose provides tools such as semantic labels and proper layout spacing to improve accessibility.

Usability Checklist

Developers evaluate UI components using the following criteria:

1. Button Size

Buttons should be at least 48dp in height according to Material Design guidelines.

Small buttons are difficult to tap and can cause user frustration.

2. Proper Spacing

Buttons should have enough space between them to prevent accidental taps.

Example:

Button A Button B

instead of

ButtonAButtonB

3. Visual Feedback

When a button is pressed, it should provide feedback such as:

- **Ripple animation**
- **Color change**
- **Highlight effect**

This confirms to the user that the action was registered.

4. Accessibility Labels

Buttons should include content descriptions for screen readers.

Example:

```
Button(
   onClick = { },
   modifier = Modifier.semantics {
      contentDescription = "Increase Counter"
   }
)
```

Accessibility ensures all users including visually impaired individuals can interact with the application. Compose provides tools such as semantic labels and proper layout spacing to improve accessibility.

Issue	Description	Fix	Priority
Small buttons	Touch difficulty	Use 48dp size	High
Poor contrast	Hard to read text	Increase contrast	Medium
No feedback	User unsure click worked	Add ripple effect	High
Missing labels	Screen readers fail	Add contentDescription	High

Importance of Accessibility

Accessibility improves the application in many ways:

- Makes the app usable by everyone
- Improves usability overall
- Meets modern design standards

Task 4
Interactive Feature Plan & Implementation Report

Task 4 is the capstone task where students plan a complete interactive feature before coding.

This task involves:

- **Designing UI layout**
- **Planning button actions**
- **Defining application states**
- **Writing pseudocode**
- **Creating a testing strategy**

This approach reflects real-world software development practices where developers plan before implementing.

Example Feature

A Quiz Application Interface.

Buttons included:

- **Next Question**
- **Submit Answer**
- **Retry Quiz**

Button Action Plan

Button	Action	State Change
Next	Move to next question	questionIndex++
Submit	Submit answer	showResult = true
Retry	Restart quiz	Reset all values

State Model

Example states of the application:

Start State
↓
Question Display
↓
Answer Submitted
↓
Result Screen
↓
Retry → Back to Start

State diagrams help developers understand how the system behaves under different conditions.

Pseudocode Example

initialize questionIndex = 0

Button Next:
questionIndex++

Button Submit:
showResult = true

Button Retry:
reset questionIndex
hide result

Test Plan

Test Case	Action	Expected Result
TC1	Click Next	Next question appears
TC2	Click Submit	Result screen appears
TC3	Click Retry	Quiz restarts

Reflection

During implementation developers may face challenges such as:

- **Handling rapid button clicks**
- **Managing state correctly**
- **Handling screen rotation**
- **Avoiding UI inconsistencies**

These challenges can be solved by:

- **Proper state management**
- **Thorough testing**
- **Modular UI design**

Button	Function	State Change
Next	Go to next item	index++
Submit	Submit answer	showResult=true
Retry	Restart process	reset state

The feature plan defines how buttons interact with application state and how the UI responds to each action.

Algorithm

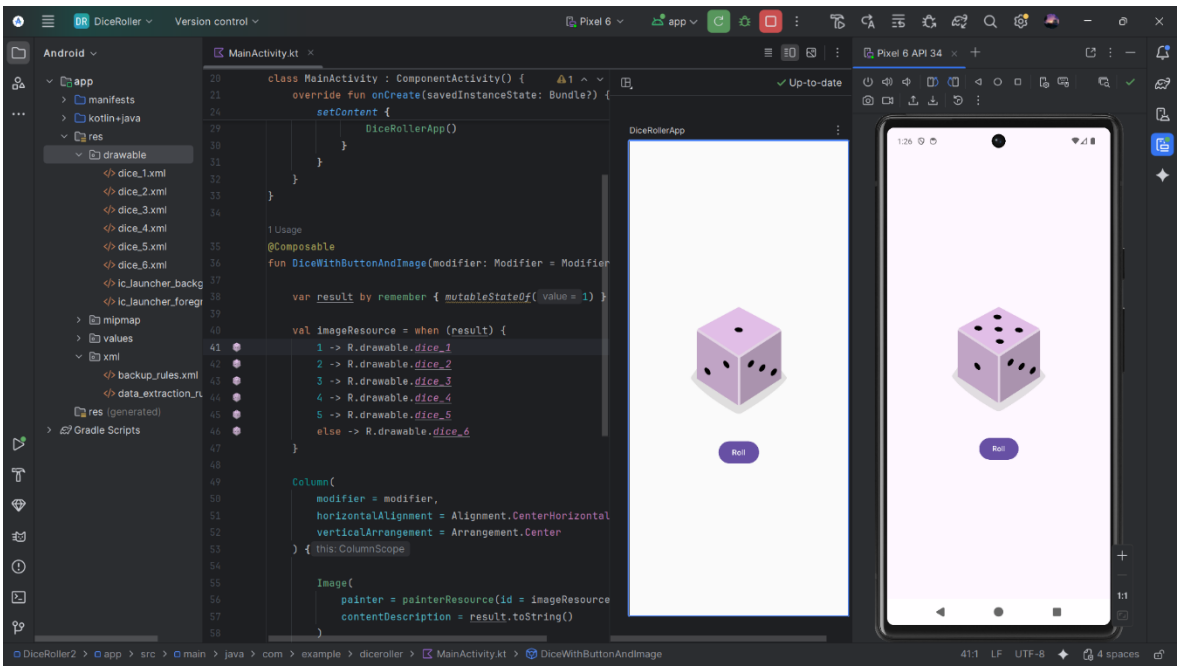
Step1 Start Application
Step2 Display UI Layout
Step3 Wait for Button
Click Step4 Execute
OnClick Logic Step5
Update State Variable
Step6 Compose Recomposition Updates UI
Step7 End

Code For Dice Roller With Buttons

```
package com.example.diceroller

import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.activity.enableEdgeToEdge
import androidx.compose.foundation.Image
import androidx.compose.foundation.layout.*
import androidx.compose.material3.Button
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.Surface
import androidx.compose.material3.Text
import androidx.compose.runtime.*
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.res.painterResource
import androidx.compose.ui.tooling.preview.Preview
import androidx.compose.ui.unit.dp

class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContent {
            Surface(
                modifier = Modifier.fillMaxSize(),
                color = MaterialTheme.colorScheme.background
            ) {
                DiceRollerApp()
            }
        }
    }
}
```

```
@Composable
fun DiceWithButtonAndImage(modifier: Modifier = Modifier) {

    var result by remember { mutableStateOf(1) }

    val imageResource = when (result) {
        1 -> R.drawable.dice_1
        2 -> R.drawable.dice_2
        3 -> R.drawable.dice_3
        4 -> R.drawable.dice_4
        5 -> R.drawable.dice_5
        else -> R.drawable.dice_6
    }

   Column(
        modifier = modifier,
        horizontalAlignment = Alignment.CenterHorizontally,
        verticalArrangement = Arrangement.Center
    ) {

        Image(
            painter = painterResource(id = imageResource),
            contentDescription = result.toString()
        )

        Spacer(modifier = Modifier.height(16.dp))

        Button(
            onClick = { result = (1..6).random() }
        ) {
            Text(text = "Roll")
        }
    }
}

@Preview(showBackground = true)
@Composable
fun DiceRollerApp() {
    DiceWithButtonAndImage(
        modifier = Modifier
            .fillMaxSize()
            .wrapContentSize(Alignment.Center)
    )
}
```

Output :



Detailed Code Explanation

1 The remember function stores state in Compose.
2 mutableStateOf creates observable state.
3 Button composable defines clickable UI element.
4 onClick lambda executes when button pressed.
5 State change triggers recomposition.
6 Text composable updates automatically.

Conclusion

This report explained Jetpack Compose button interaction and UI state management. By dividing the report into tasks we analysed UI interaction, design planning, accessibility evaluation, and feature planning.

References and Bibliography

1 Android Developers Jetpack Compose Documentation
2 Kotlin Programming Language Documentation
3 Android UI Design Guidelines
4 Material Design Guidelines